

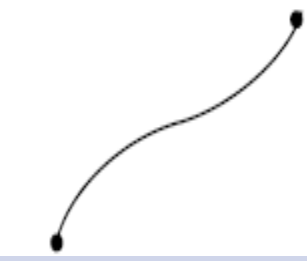
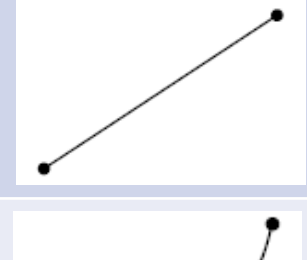
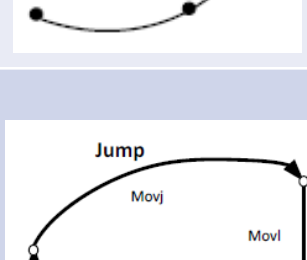
프로그램 기초
주요 명령어

3

주요 명령어

PART 1

이동 명령어를 사용하여 로봇을 원하는 위치로 이동시킬 수 있습니다. 이동하는 경로의 궤적을 각 명령어들을 통해 변경할 수 있습니다.

명령어	이동 경로	설명
MovJ (Joint motion)		Joint 움직임으로 P[1]에서 P[2]로 이동합니다. 이동중에 직선 보간 제어를 하지 않아 이동 속도가 가장 빠릅니다.
MovAbsJ (Joint motion)		Joint 움직임으로 JP[n] 위치값을 사용합니다. MovJ와 동일한 모션을 수행하지만 MovJ와 큰 차이점은 위치 값이 좌표계가 아닌 각 축의 각도(Joint)를 기준으로 기동하여 특이점 자세로도 이동이 가능합니다.
MovL (Linear motion)		P[1]에서 P[2]로 직선 보간 제어합니다. 목표 위치까지 일직선으로 이동합니다.
MovC (Arc motion)		P[1] - P[2] - P[3] 세 점의 위치를 참조하여 원호 보간 제어합니다. 중간점을 잘못 티칭하거나 원호가 그려지지 않을 때는 에러가 발생합니다.
Jump(L) (Jump motion)		Scara 로봇에서만 사용할 수 있는 이동 명령어 그림과 같은 경로를 구현하기 위해서는 4개의 포인트가 필요하지만 Jump 명령어를 사용하여 2포인트로 이동이 가능 Movl 경로로 Z축을 상승시키고 Movj 경로로 P[2]위치 위에 도달 후 Movl 경로로 Z축을 하강합니다. JumpL 명령어는 Movj 경로가 Movl 경로로 변경됩니다.

Movj P[n],V[n](or Speed[n]),Z[n](or Zr[n]),Tool[n],Wobj[n];

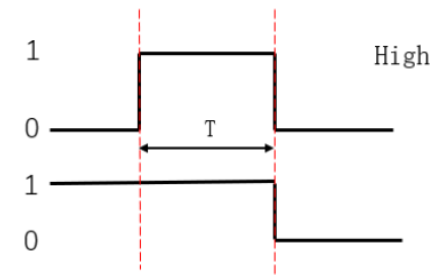
기본 파라미터	설명
P[n]	이동할 목표 위치(point)를 설정합니다. (P[1], LP[1], P[B[1]], LP[B[1], Label 등)
V[n]	이동 속도를 설정합니다. (1~100, 속도 기준은 로봇 파라미터 설정에서 설정된 최대 속도 * 오버라이드% * V[n]%)
Speed[n]	이동 속도를 mm/s 단위로 입력합니다. (MovJ 사용 불가)
Z[n]	Zone, 목표 지점을 설정값 만큼 도달하지 않고 다음 이동 경로로 이동하여 연속적인 움직임을 생성합니다. (설정 Corner 값 * Z[n]) (0~200, Fine, CP)
Zr[n]	Zone, 목표 지점을 설정값 만큼 도달하지 않고 다음 이동 경로로 이동하여 연속적인 움직임을 생성합니다. (100은 전체 경로 절반이며, Z[CP]와 동일한 영역을 의미합니다.
Tool[n]	P 변수에 저장되어 있는 위치값을 어느 TCP를 기준으로 이동할지 결정합니다.
Wobj[n]	P 변수에 저장되어 있는 위치값을 어느 Work obj 좌표를 기준으로 이동할지 결정합니다.

추가 형식 : Movj P[n], V[n], Z[n]... , Acc[n], Dec[n], NWait, Until IN[n]==ON, Out(Out[0], ON), SLOn ...;

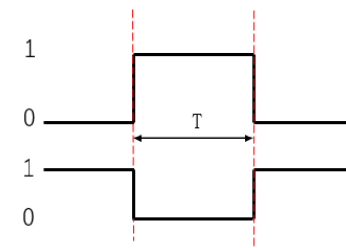
기본 파라미터	설명
Acc[n], Dec[n]	해당 이동 경로의 가/감속을 설정합니다. (1~120)
Nwait	이동 명령이 실행되면 해당 명령 다음 행의 비이동 명령들을 이동 명령 실행과 동시에 실행합니다.
Until In[n] == ON/OFF	ON or OFF : 설정한 In[n]의 값이 On되거나 Off 될 경우에 진행 중인 이동 명령을 중지합니다. (In[0] ~ In[63])
Out(Out[n], On/Off, T[n])	Out(0~63, ON or OFF, T[n] or Ds[n] or S[n]) : 이동 중 Out 신호를 원하는 지점에서 On/Off 합니다. T[n] : T[n] > 0 = 명령 시작 n초 후 신호 출력, T[n] < 0 = 목표 지점 도착하기 n초 전에 신호 출력 Ds[n] : Ds[n] > 0 = 명령 시작 n% 지점에서 신호 출력, Ds[n] < 0 = 목표 지점 도착하기 n% 이전에 신호 출력 S[n] : S[n] > 0 = 명령 시작 'n' mm 지점에서 신호 출력, S[n] < 0 = 목표 지점 도착하기 'n' mm 이전에 신호 출력
SLOn	자가 학습 진동억제 기능 켜, 해당 경로 이동 시 발생한 진동을 억제하기 위한 알고리즘이 실행되며 해당 진동을 억제하기 위한 모션 경로 데이터를 자동으로 생성하여, 다음 똑같은 경로로 이동 시 진동 억제된 모션 경로로 이동합니다.
SLOff	진동 억제 모션 경로를 사용하지 않고 기본 경로로 이동합니다.
SLReset	저장한 진동 억제 모션 경로를 삭제합니다.
Offset(P[n], PR[n])	목표 위치인 P[n]의 PR 변수의 위치 값이나, X, Y, Z 등의 좌표 위치 값을 더해서 목표 위치를 변경할 수 있습니다.
Pallet(No, R, C, L)	Pallet(Pallet No, Row, Column, Lay): 설정된 팔레타이징 위치로 이동합니다. 앞서 Pallet 명령어를 사용하여 Pallet 변수를 정의해야 합니다.

명령어	형식	설명
Set	Set Out[n], ON/OFF;	DO를 On/Off 합니다.
	n: 0~13823	
Get	Get IN/OUT[n], Var;	DI/O의 현재 값을 읽고 Var에 저장합니다.
	N: 0~13823 Var: B, R, D, User ETC..	
Pulse	Pulse Out[n], T[s], High;	DO의 현재 값을 설정 시간동안 반전 시킨 뒤, 원래의 값으로 되돌아갑니다. 만약 High를 입력 시 반전이 아닌, ON 상태를 유지시킨 뒤, OFF시킵니다.
	n: 0~13823 s: 시간(0.1~2000초)	
Group	Group IG(OG)[n], n0, n1, ..., n7;	1BIT의 DI/O를 최대 8bit까지 하나의 그룹으로 묶을 수 있습니다. 해당 그룹은 여러 개의 bit의 조합을 byte 형식으로 나타냅니다.
	[n]: 그룹 변수(0~15) n: DI/O (0~127)	

1. Pulse Out[1],T[10],High; #Sets Out[1] to output ON, and OFF after 10s



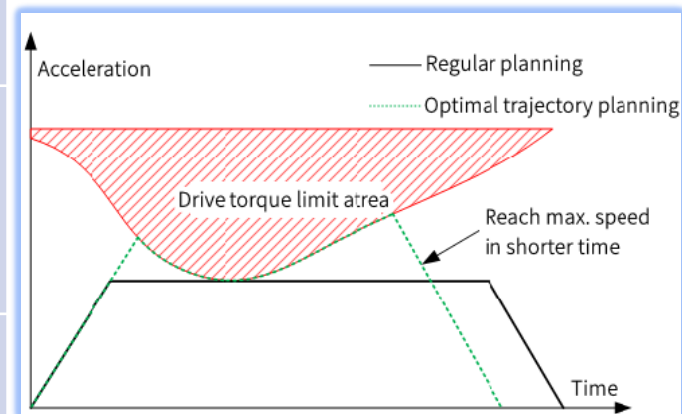
2. Pulse Out[1],T[10]; #Sets Out[1] to output state opposite to the current state, and the original state after 10s



*만약 전압 레벨이 반대되는 명령이 2개가 겹칠 경우 기존 명령은 무시되며, 가장 아래(가장 최근)에 실행된 명령만 실행됩니다.

명령어	형식	설명
SetAccRamp	SetAccRamp(StartVal,EndVal);	로봇의 기동 가(StartValue)/감속(EndValue)의 저크값을 설정합니다. 해당 명령어가 실행된 이후 모든 모션에 적용됩니다. 사용하지 않는 경우 로봇 모델별 기본 저크값으로 설정됩니다.
	StartVal: 1~100% EndVal: 1~100%	
Velset	Format 1: Velset Rate[value]; Format 2: Velset [value];	로봇의 기동 속도를 설정합니다. 해당 명령어가 실행된 이후 모든 모션에 적용됩니다. Velset Rate: 전체 속도(오버라이드)를 변경합니다. Velset: 이동 명령어의 'V'값을 무시하고 해당 속도로 변경합니다. 이 명령이 실행되고 나서는 모든 이동 명령의 'V'값은 무시되며, Velset OFF 명령 이후에 정상화됩니다.
	Value = 1~100%, OFF	
SetAcc	SetAcc(accVal,DecVal);	로봇의 기동 가감속도를 설정합니다. 해당 명령어가 실행된 이후 모든 모션에 적용됩니다. 해당 명령어각 실행되면 이동 명령어 내에 있는 가/감속 값은 무시됩니다. SetAcc OFF 명령 이후에 정상화됩니다.
	accVal: 1~120%, OFF decVal: 1~120%, OFF	
RapidMove	RapidMove(Type, Enable)	로봇의 가감속도를 자동으로 설정합니다. (SetAccRamp, SetAcc의 값 적용됨) 자동으로 현재 로봇의 자세에 따라 낼 수 있는 최대의 가감속도로 기동합니다. Tool이나 Load 설정을 하지 않으면(최대 부하를 적용하여 느린 속도로 기동합니다)
	Type: CP, PTP, All (CP 미지원) Enable: ON, OFF	
SLVSMODE	SLVSMODE Level;	이동 명령어의 SLOn이 적용된 자가학습 진동억제 기능이 사용되었을 경우 진동 억제 레벨을 설정합니다. 진동 억제 정도를 높게 설정하면 진동 억제 수준이 상승하고 속도가 저하됩니다.
	Level: High, Mid, Low level, OFF	
		진동 억제 정도 : High >= Mid >= Low 속도 : Low >= Mid >= High

```
#전체 속도 (오버라이드) 속도를 100% 설정
Movj P[1],V[70],Z[0],Tool[0]; #실제 속도: 100%*70%=70%
Velset Rate[50]; #오버라이드 속도 50%로 변경
Movj P[2],V[70],Z[0],Tool[0]; #실제 속도: 50%*70%=35%
Velset 30;
Movj P[3],V[70],Z[0],Tool[0]; #실제 속도: 50%*30%=15%
subporg.Func1(); #서브 프로그램은 해당 명령어 적용되지 않음
Velset OFF;
Movj P[4],V[70],Z[0],Tool[0]; #실제 속도: 50%*70%=35%
```



명령어	형식	설명
L	L[Label No]:	라벨 아래의 프로그램 행을 라벨로 구분 짓습니다.
	Label No: 라벨 넘버	
Goto L	Goto[Label No];	해당 라벨 행으로 점프합니다.
	Label No: 이동할 라벨	
Delay	Delay T[n];	다음 행을 처리하기까지 시간을 지연시킵니다. 초 단위로 설정합니다. (0.000~65535.000초)
	n: 지연시킬 시간 (0.000~65535.000)	
#Commnet	#Any Text	'#'를 삽입한 이후 텍스트는 주석 처리됩니다. 주석은 초록색으로 표시됩니다.
	-	

```

1   Start;
2   #Test Program 1
3   L[1]: #Label 1
4   Movj P[1],V[30],Z[0],Tool[0];
5   Goto L[5];
6
7   L[2]:
8   Delay T[1.5];
9   Goto L[5];
10
11  L[5]:
12  Movj P[2],V[30],Z[0],Tool[0];
13  Goto L[1];
14  End;
    
```

명령어	형식	설명
Include	Include "Module.pro";	다른 모듈의 Func을 호출할 경우 해당 Func이 쓰여진 프로그램을 참조해야 합니다. 해당 명령으로 호출할 Func의 프로그램을 참조할 수 있습니다.
	"Prog": 참조할 모듈 이름	
Func	Func FuncName (Param)	해당 명령 안에 쓰여진 프로그램을 함수로 사용합니다. 다른 프로그램에서 불러와 사용하거나 같은 프로그램 안에서 사용 가능합니다.
	FuncName:	
Ret	Ret	Func 프로그램 안에 Ret 명령어가 존재하면 Ret 명령어가 실행될 때 Func 프로그램이 종료되고 main 프로그램으로 되돌아갑니다.
	-	
Return	Return Var;	Func 프로그램의 결과값을 반환하고, 설정 변수에 저장합니다.
	Var: B, R, D, User, ETC	

```
Include "Test.pro";
Start;
    Bool Var;
    Test.func1();
    If Var == 1
        Print B[0];
    EndIf;
End;
```

```
Func func1() #Test.pro
    If B[0] == 0
        Bool Var = True;
    Else
        Ret;
    EndIf;
    Return Var;
EndFunc;
```

명령어	형식	설명
IF	If condition	특정 조건이 참인 경우 IF문 안에 프로그램을 실행합니다. Condition에 조건 변수와 비교문을 입력하여 사용할 수 있습니다.
	Condition: 조건 변수	
Elseif	Elseif condition	IF 조건이 거짓이고 Elseif의 조건이 참이면 Elseif의 프로그램을 실행합니다.
	Condition: 조건 변수	
Else	Else	IF, Elseif 조건이 모두 거짓이면 Else 안의 프로그램이 실행됩니다.
	-	
EndIf	EndIf;	IF문을 닫습니다.
	-	

```

Start;

    If In[50] == ON And In[10] == OFF

        Print "Run";

    ElseIf B[1] >= 16

        Print "Stop";

    Else

        Pause

    EndIf;

End;
    
```

명령어	형식	설명
For	For InitExp, Condition, Step[n]	For문 안에 프로그램을 반복 실행합니다. For문을 처음 실행하면 InitExp을 실행하며, 반복할 때 마다 Condition 조건 변수에 Step[n]번 증가시킵니다. Ex) For i=0, i<5, Step[2] = i는 0으로 설정하며, i가 5 미만이면 계속 반복, 반복하면 i를 +2 증가시킵니다.
	InitExp: 초기화 변수 (B, R) Condition: 반복 조건 Step[n]: 반복 카운트	
EndFor	EndFor;	For문을 닫습니다.
	-	

```
Start;
    int i;
    For i=1,i<=5,Step[1]
        Print "Cur count : "+i;
    EndFor;
End;
```

명령어	형식	설명
While	While Condition	While Condition이 참일 경우 While문 안에 프로그램을 반복 실행합니다.
	Condition: 반복 조건	
EndWhile	EndWhile;	While문을 닫습니다.
	-	

```
Start;
    int i;
    While i<=5
        Print "Cur count : "+i;
        Incr i;
    EndWhile;
End;
```

명령어	형식	설명
Break	Break;	반복문 안에 Break가 있다면 반복문을 빠져나옵니다.
	-	
Continue	Continue;	반복문 안에 Continue가 있다면 Continue 아래 행의 프로그램은 실행하지 않고 반복문 처음으로 되돌아갑니다.
	-	

```

i = 0;
For i=0,i<5,Step[1]
  If i > 1
    Break;
  EndIf;
  Movj P[1],V[30],Z[0],Tool[0],Wobj[0];
  Movj P[2],V[30],Z[0],Tool[0],Wobj[0];
EndFor;
    
```

<Break>

```

i = 0;
For i=0,i<5,Step[1]
  If i > 1
    Continue;
  EndIf;
  Movj P[1],V[30],Z[0],Tool[0],Wobj[0];
  Movj P[2],V[30],Z[0],Tool[0],Wobj[0];
EndFor;
    
```

<Continue>

명령어	형식	설명
Switch	Switch Var	Case 조건이 되는 변수를 설정합니다. (Integer or String)
	Var: Case 조건이 되는 정수 변수	
Case	Case Value:	Switch Var의 값이 Case Value와 동일할 경우 해당 Case문 안에 프로그램을 실행합니다.
	Value: Switch Var의 조건값	
Default	Default:	Switch Var의 값이 Case 조건에 없을 경우, Default 안에 프로그램을 실행합니다.
	-	
EndSwitch	EndSwitch;	Switch – Case 문을 닫습니다.
	-	

```
Switch B[0]
  Case 0:
    Print "Cur case" + B[0];
    Break;
  Case 1 to 10:
    Print "Cur case" + B[0];
    Break;
  Default:
    Print "Cur case" + B[0];
    Break;
EndSwitch;
```

명령어	형식	설명
Incr	Incr Var;	설정한 Var에 값을 +1하여 다시 저장합니다.
	Var: B, LB, R, LR, Int, Byte var	Ex) B[1] = 1; Incr B[1]; ## B[1] == 2;
Decr	Decr Var;	설정한 Var에 값을 -1하여 다시 저장합니다.
	Var: B, LB, R, LR, Int, Byte var	Ex) B[1] = 1; Decr B[1]; ## B[1] == 0;
Max	Var = Max (x, y);	X, Y의 숫자를 비교하여 더 큰 숫자의 값을 Var에 저장합니다.
	Var: 숫자 변수 x, y: 비교 대상	Ex) B[1] = 1; B[2] = 2; D[1] = Max (B[1], B[2]); ## D[1] == 2
Min	Var = Min (x, y);	X, Y의 숫자를 비교하여 더 작은 숫자의 값을 Var에 저장합니다.
	Var: 숫자 변수 x, y: 비교 대상	Ex) B[1] = 1; B[2] = 2; D[1] = Min (B[1], B[2]); ## D[1] == 1

명령어	형식	설명
Dist	Var = Dist(P1, P2);	시작과 끝 위치 사이 직선 거리를 계산하여 Var 변수에 저장합니다. *시작 위치 변수의 좌표계를 기준으로 거리값이 저장됩니다. *P 변수만 사용가능합니다.
	Var: 숫자 변수 P1: 시작 위치 P2: 끝 위치	
GetCurPos GetCurJPos	P[*] = GetCurPos(Tool[n], Wobj[n]); JP[*] = GetCurJPos();	현재 TCP, Workobj의 좌표값을 설정한 P(JP) 변수에 저장합니다.
	P[*]: 값을 저장할 P 변수 JP[*]: 값을 저장할 JP 변수	
P = Offset	P[*]=Offset(P[n],PR[n]); P[*]=Offset(P[n],Z(X,Y)[n]);	P[n]의 위치 값 + PR[n]을 offset하여 P[*] 변수에 저장합니다. PR 변수 외에도 X, Y, Z 값을 직접 입력하여 offset할 수 있습니다. Ex) P[1]=Offset(P[1], PR[1]); #PR 변수 활용 P[2]=Offset(P[1], X[50], Y[-30], Z[10]); #offset 직접 입력
	P[*]: offset된 위치 값 저장 P[n]: 기준 위치 값 PR[n]: Offset할 변수 Z or X or Y [n]: Offset 직접 입력	
Pallet	Pallet No, P[1], P[2], P[3], Row, Col, Lay, LayHeight;	Pallet No에 팔레트 정보를 선언합니다. *Row, Col, Lay 개수는 1~1000까지 선언 가능합니다. *LayHeight 높이는 0~2000mm까지 선언 가능합니다. Ex) Pallet 1, P[10], P[11], P[12], 5, 2, 2, 15;
	Pallet No: Pallet 변수 (0~255) P[1], P[2], P[3]: Pallet 기준 위치 Row, Col, Lay: Pallet 가로, 세로, 층 개수 지정 LayHeight: 층 높이	

```
Start;
  D[0] = Dist(P[0],P[1]);
  Print D[0];
END;
```

```
Start;
  LP[0] = GetCurPos();
  LP[1] = GetCurPos(Tool[0],Wobj[0]);
  Print LP[0];
  Print LP[1];
END;
```

```
Start;
  P[100] = Offset(P[0],PR[1]);
  P[101] = Offset(P[0],X[10],Y[10],Z[-10]);
END;
```

```
Pallet 0,P[0],P[1],P[2],3,2,2,10;
#팔레트 0번 변수의 2번째 행, 1번째 열, 1번째 레이어 위치로 이동
Movj Pallet(0,1,0,0),V[30],Z[0],Tool[0],Wobj[0];
```

명령어	형식	설명
Alarm	Alarm [n], EasyGo;	설정한 유저 알람을 띄워 알람을 생성합니다. EasyGo 명령을 추가하면 다음 행으로 커서를 옮깁니다. 에러를 해제하고 실행하면 다음 라인부터 실행할 수 있습니다.
	Alarm [n]: 유저 알람 번호(0~15)	
Print	Print object;	글자나 변수들의 값을 알림창에 그대로 출력합니다. *사칙연산 불가 Ex) Print B[1] + B[2] #B[1] = 5, B[2] = 6 = 5 6
	Object: Text, P, B, R, D 등 변수	
TimeStart	TimeStart(n);	해당 명령어가 실행되면 그때부터 시간을 측정합니다. (스톱워치) 0~99까지 별개의 스톱워치를 실행시킬 수 있습니다.
	n: time index (0~99)	
TimeOut	TimeOut(n, Var);	해당 명령어가 실행되면 설정한 스톱워치의 time index 시간 측정을 종료하고 Var 변수에 시간 값을 저장합니다. (unit: sec)
	n: Time index (0~99) Var: D	

```

1 Start;
2     Delay T[1];
3     Alarm[0], EasyGo;
4     Delay T[1];
5 END;
```

```

Start;
    B[0] = 2;
    Print "B[0]의 값은 " + B[0] + " 입니다";
END;
```

Output

Message: 17:06:07 successfully set all tasks to return to the start time

[Print]: 17:06:08 B[0]의 값은 2 입니다

```

Start;
    TimeStart(0);
    Delay T[1.52];
    TimeOut(0, D[0]);
    Print D[0];
END;
```

Output

Message: 17:07:18 1.520

명령어	형식	설명
GripLoad	GripLoad ObjNo;	해당 명령어가 실행되면 Grip Load의 값을 변경합니다. Grip Load에 경우 Load 파라미터를 미리 저장해야 합니다. Ex) GripLoad 1;
	ObjNo: Load No	

```
Start;  
  GripLoad 1;  
END;
```

프로그램 예제

PART 2

```

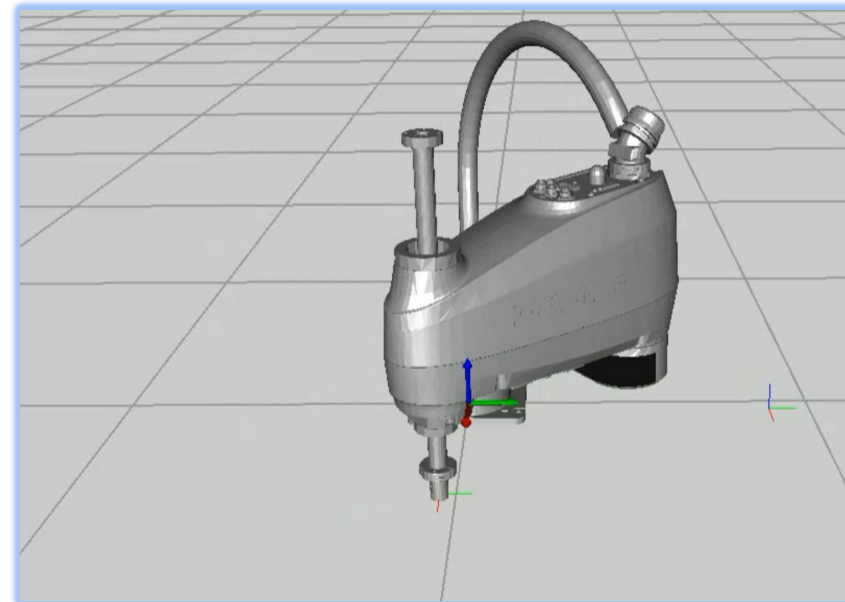
1  #MovJ/L 명령을 사용한 Pick & Place 프로그램
2  Start;
3  L[0]:
4  If In[0] == ON #동작 시작 입력
5      Set Out[0],ON; #사이클 동작 시작 출력
6      Movj P[0],V[100],Z[0],Tool[0],Wobj[0]; #P[0] Pick 대기 위치로 이동
7      Movl Offset(P[0],Z[-10]),V[30],Fine,Tool[0],Wobj[0]; #Pick 위치로 이동
8      Set Out[1],ON; #흡착 ON
9      Delay T[0.5]; #0.5초간 대기
10     Movl Offset(P[0],Z[10]),V[30],Z[1],Tool[0],Wobj[0]; #Z축 10만큼 수직 이동
11     Movj P[1],V[100],Z[0],Tool[0],Wobj[0]; #P[1] Place 대기 위치로 이동
12     Movl Offset(P[1],Z[-10]),V[30],Fine,Tool[0],Wobj[0]; #Place 위치로 이동
13     Set Out[1],OFF; #흡착 OFF
14     Movj P[10],V[50],Z[0],Tool[0],Wobj[0]; #대기 위치로 이동
15     Set Out[0],OFF; #1사이클 동작 완료
16 EndIf;
17 Goto L[0]; #라벨 0으로 이동하여 프로그램 반복 구성
18 END;

```

```

1  #Jump 명령을 사용한 Pick & Place 프로그램
2  Start;
3  L[0]:
4  If In[0] == ON #동작 시작 입력
5      Set Out[0],ON; #사이클 동작 시작 출력
6      Jump P[0],V[100],Fine,Tool[0],Wobj[0],LH[10],MH[-50],RH[10]; #Pick 위치로 이동
7      Set Out[1],ON; #흡착 ON
8      Delay T[0.5]; #0.5초간 대기
9      Jump P[1],V[100],Fine,Tool[0],Wobj[0],LH[10],MH[-50],RH[10]; #Place 위치로 이동
10     Set Out[1],OFF; #흡착 OFF
11     Jump P[10],V[100],Z[0],Tool[0],Wobj[0],LH[10],MH[0],RH[0]; #대기 위치로 이동
12     Set Out[0],OFF; #1사이클 동작 완료
13 EndIf;
14 Goto L[0]; #라벨 0으로 이동하여 프로그램 반복 구성
15 END;

```



```

1  #조건문으로 각 다른 Func(함수) 실행 프로그램
2  Include "test.pro";
3  Start;
4  L[0]:
5  If In[0] == ON #동작 시작 입력
6      test.func1(); #test 프로그램의 func1 실행
7  ElseIf In[0] == OFF and In[1] == ON
8      # In[0]은 OFF고 In[1]은 ON되어 있을 경우
9      test.func2(); #test 프로그램의 func2 실행
10 Else #위 두개 조건이 모두 거짓일 경우
11     test.func3(); #test 프로그램의 func3 실행
12 EndIf;
13 Delay T[1];
14 Goto L[0]; #라벨 0으로 이동하여 프로그램 반복 구성
15 END;
    
```

```

test.pro x main.pro
1  Func func1() #test 프로그램
2      Set Out[0],ON; #사이클 동작 시작 출력
3      Jump P[0],V[100],Fine,Tool[0],Wobj[0],LH[10],MH[0],RH[10]; #Pick 위치로 이동
4      Set Out[1],ON; #흡착 ON
5      Delay T[0.5]; #0.5초간 대기
6      Jump P[1],V[100],Fine,Tool[0],Wobj[0],LH[10],MH[0],RH[10]; #Place 위치로 이동
7      Set Out[1],OFF; #흡착 OFF
8      Jump P[10],V[100],Fine,Tool[0],Wobj[0],LH[10],MH[0],RH[10]; #대기 위치로 이동
9      Set Out[0],OFF; #1사이클 동작 완료
10 EndFunc;
    
```

```
1  #스위치-케이스로 각 다른 Func(함수) 실행 프로그램
2  Include "test.pro";
3  Start;
4      Group IG[0],0, 1;
5      Get IG[0],B[0];
6      L[0]:
7      Switch B[0]
8      case 1: #IG[0]의 값이 1일 경우
9              test.func1(); #test 프로그램의 func1 실행
10             Break;
11     case 2: #IG[0]의 값이 2일 경우
12             test.func2(); #test 프로그램의 func2 실행
13             Break;
14     Default: #IG[0]의 값이 위 조건과 일치하지 않는 모든 경우
15             Alarm[0],EasyGo; #유저 알람 0번 발생
16             Break;
17     EndSwitch;
18     Delay T[1];
19     Goto L[0]; #라벨 0으로 이동하여 프로그램 반복 구성
20 END;
```

```
9 Start;  
10 L[0]:  
11 B[0] = 0;  
12 If B[0] < 5  
13     Movj P[0],V[30],Z[0],Tool[0],Wobj[0];  
14     Movj P[1],V[30],Z[0],Tool[0],Wobj[0];  
15     Movj P[2],V[30],Z[0],Tool[0],Wobj[0];  
16     Movj P[3],V[30],Z[0],Tool[0],Wobj[0];  
17     Movj P[4],V[30],Z[0],Tool[0],Wobj[0];  
18     Movj P[5],V[30],Z[0],Tool[0],Wobj[0];  
19     Movj P[6],V[30],Z[0],Tool[0],Wobj[0];  
20     Incr B[0];  
21     Goto L[0];  
22 EndIf;  
23 End;
```

```
1 Start;  
2     B[0] = 0;  
3 While B[0] < 5  
4     Movj P[B[0]],V[30],Z[0],Tool[0],Wobj[0];  
5     Incr B[0];  
6 EndWhile;  
7 End;
```

```
1 Start;  
2 For B[0]=0,B[0]<5,Step[1]  
3     Movj P[B[0]],V[30],Z[0],Tool[0],Wobj[0];  
4 EndFor;  
5 End;
```

INOVANCE

Forward, Always Progressing